

Actividad 4: Redes de Datos, Seguridad y Sistemas Distribuidos

Fernando Israel Rios Garcia-AL02877016@tecmlenio.mx

Artur Mikhail Lazurenko Mannanov-AL03028732@tecmlenio.mx

Angel Cuevas Lopez-AL03033114@tecmlenio.mx

Universidad Tecmlenio

LSTI2308: Sistemas Operativos

Ana Cruz

22 de febrero de 2026

Índice

1. Introducción	4
2. Desarrollo	4
2.1. Parte 1. Diseño y configuración de una red de datos	4
2.1.1. Implementación: Diseño de Red de Datos Básica	4
2.1.2. Esquema de la red	6
2.1.3. Modelo OSI Aplicado al Diseño	7
2.1.4. Flujo de Comunicación	9
2.2. Parte 2. Aplicación de medidas de seguridad	9
2.2.1. Configuración de firewalls	9
2.2.2. Implementación de autenticación segura	10
2.2.3. Configuración de HTTPS	11
2.2.4. Análisis de vulnerabilidades y privacidad	11
2.3. Parte 2. Aplicación de medidas de seguridad	13
2.3.1. Configuración de Firewall (UFW)	13
2.3.2. Hardening de SSH (Autenticación Criptográfica)	14
2.3.3. Prevención de Intrusiones (Fail2ban)	16
2.3.4. Cifrado HTTPS (Certificados Autofirmados)	16
2.3.5. Configuración Inicial	16
2.3.6. Sistema de Archivos Distribuido (NFS)	17
2.3.7. Red Peer-to-Peer (BitTorrent)	20
2.3.8. Reflexión sobre sistemas distribuidos	26
3. Análisis y reflexión	28
4. Conclusiones	29

Índice de figuras

1.	Configuración de interfaz de red y asignación de direcciones IP en la máquina virtual.	6
2.	Comprobación de conectividad mediante ping (Capa 3 de red).	7
3.	Respuesta exitosa de ping, conectando exitosamente las VM en Bridge UTM.	8
4.	Listado de estado activo y reglas restrictivas vigentes en UFW.	10
5.	Fragmento de demostración tras revisión de vulnerabilidades aplicadas a la IP local ejecutando comandos Nmap.	13
6.	Comprobación de reglas activas y puertos de tráfico permitidos en el firewall UFW.	15
7.	Copia exitosa de la llave pública asimétrica RSA hacia el servidor.	31
8.	Modificación del archivo maestro <code>sshd_config</code> para prohibir las contraseñas planas.	32
9.	Login remoto de inmediato y sin contraseñas validando el acceso a través de la llave SSH.	33
10.	Servicio Fail2ban activo, monitorizando activamente la seguridad del puerto local SSH.	34
11.	Generación de claves y certificados X.509 autofirmados vía línea de comandos con OpenSSL.	35

1. Introducción

El presente reporte detalla el desarrollo de la Actividad 4, que tiene como objetivo principal implementar, configurar y asegurar una red de datos básica, al igual que explorar los sistemas distribuidos mediante la compartición de recursos en un ecosistema virtual. A lo largo de la actividad, se desplegó un entorno compuesto por dos máquinas virtuales Linux conectadas entre sí para observar en la práctica los conceptos del modelo OSI, la configuración manual de direcciones IP estáticas y la habilitación de servicios de red indispensables (como SSH y HTTP).

Posterior a la integración de la red, se aplicaron medidas de seguridad informática estipuladas como estándar en la industria. Esto se logró mediante la configuración de firewalls de host restrictivos con UFW, el uso de criptografía de clave asimétrica para el acceso SSH remoto seguro, la implementación de cifrado HTTPS con certificados autofirmados y el análisis de vectores de posibles vulnerabilidades utilizando herramientas de auditoría perimetral como Nmap. Para terminar, se configuró un sistema de archivos remoto distribuido de red local (NFS) y se exploró de forma análoga el uso práctico de protocolos P2P como BitTorrent para compartir archivos. Integrar los conocimientos de redes, resiliencia y sistemas distribuidos ayuda a consolidar una visión holística sobre la arquitectura tecnológica de las operaciones informáticas actuales.

2. Desarrollo

2.1. Parte 1. Diseño y configuración de una red de datos

2.1.1. *Implementación: Diseño de Red de Datos Básica*

Objetivo: Crear una red básica donde dos máquinas virtuales Linux puedan comunicarse entre sí utilizando VirtualBox.

Pasos:

1. Crear las máquinas virtuales

- Instalar VirtualBox.

- Crear dos máquinas virtuales llamadas: Linux VM A y Linux VM B.
- Instalar una distribución Linux en cada una (por ejemplo Ubuntu Server o Debian).

2. Configurar la red

- Ir a Configuración → Red.
- Seleccionar “Adaptador 1”.
- Elegir el modo: → “Red interna” o → “Adaptador solo-anfitrión (Host-Only)”.
- Asegurarse de que ambas máquinas estén en la misma red virtual.

3. Asignar direcciones IP manualmente

- En VM A:

```
sudo ip addr add 192.168.100.10/24 dev enp0s3
sudo ip link set enp0s3 up
```

- En VM B:

```
sudo ip addr add 192.168.100.20/24 dev enp0s3
sudo ip link set enp0s3 up
```

4. Probar conectividad

- Desde VM A:

```
ping 192.168.100.20
```

- Si responde, la red está funcionando correctamente.
- También se puede probar: **ssh**, servidor web, transferencia de archivos.

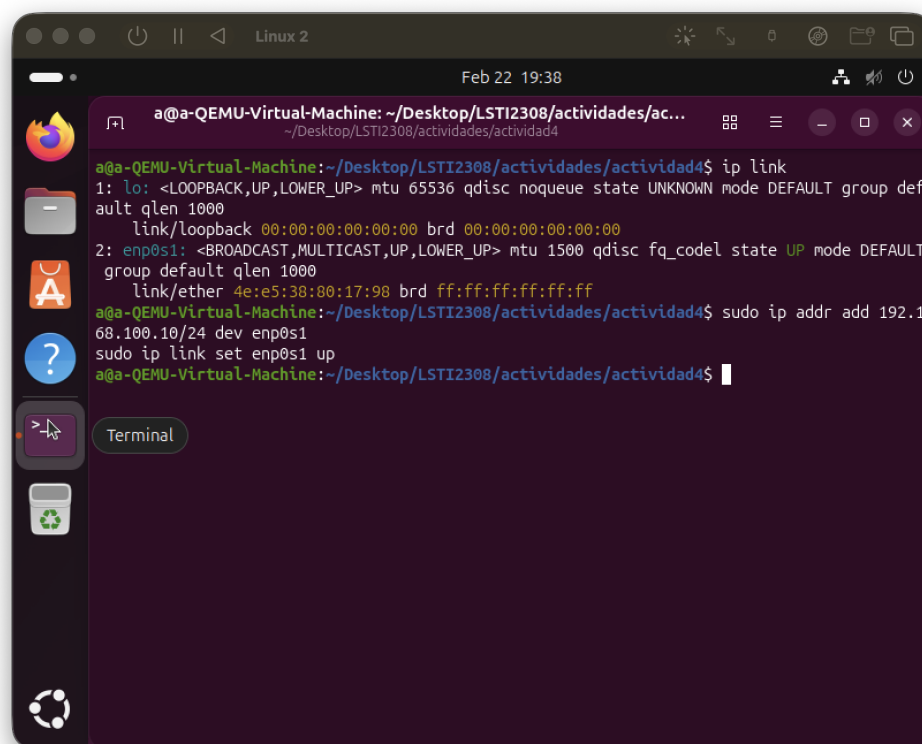


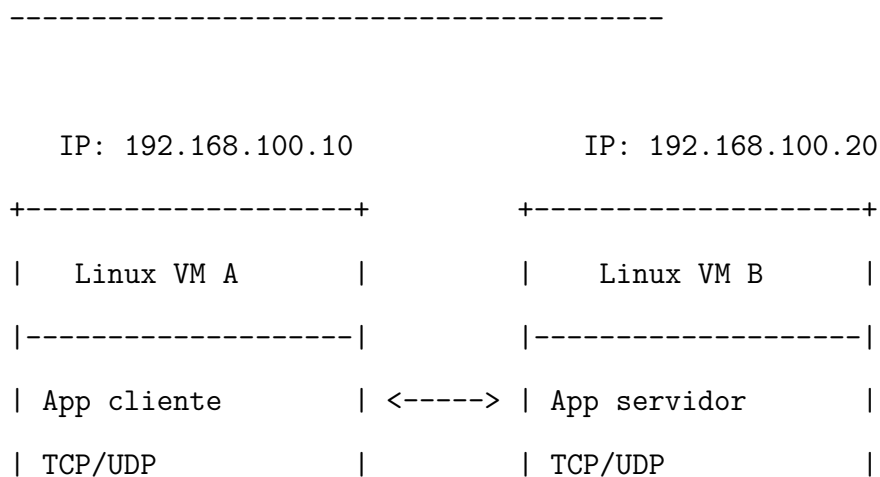
Figura 1

Configuración de interfaz de red y asignación de direcciones IP en la máquina virtual.

2.1.2. Esquema de la red

Red virtual (Host-Only / Interna)

192.168.100.0/24



```

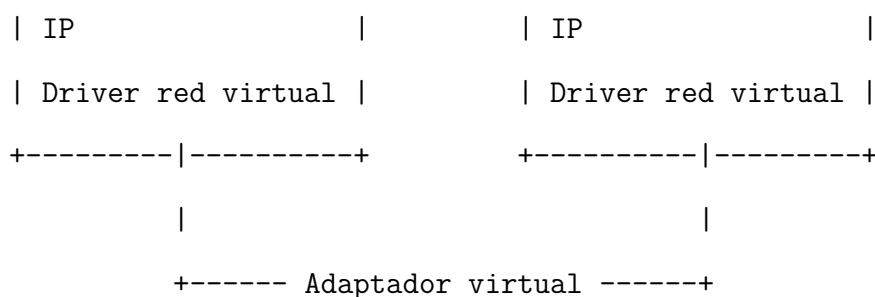
a@a-QEMU-Virtual-Machine: ~$ cd Desktop/LSTI2308/
a@a-QEMU-Virtual-Machine:~/Desktop/LSTI2308$ cd actividades/actividad4/
a@a-QEMU-Virtual-Machine:~/Desktop/LSTI2308/actividades/actividad4$ sudo nano /etc/netplan/
/00-installer-config.yaml
[sudo: authenticate] Password:
a@a-QEMU-Virtual-Machine:~/Desktop/LSTI2308/actividades/actividad4$ sudo netplan apply
systemd-networkd is not running, output might be incomplete.
Failed to reload network settings: Unit dbus-org.freedesktop.network1.service not found.
Falling back to a hard restart of systemd-networkd.service
a@a-QEMU-Virtual-Machine:~/Desktop/LSTI2308/actividades/actividad4$ sudo netplan apply
a@a-QEMU-Virtual-Machine:~/Desktop/LSTI2308/actividades/actividad4$ ping -c 4 192.168.56.1
0
PING 192.168.56.10 (192.168.56.10) 56(84) bytes of data:
64 bytes from 192.168.56.10: icmp_seq=1 ttl=64 time=1.73 ms
64 bytes from 192.168.56.10: icmp_seq=2 ttl=64 time=1.30 ms
64 bytes from 192.168.56.10: icmp_seq=3 ttl=64 time=1.41 ms
64 bytes from 192.168.56.10: icmp_seq=4 ttl=64 time=0.779 ms

--- 192.168.56.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3009ms
rtt min/avg/max/mdev = 0.779/1.303/1.725/0.340 ms
a@a-QEMU-Virtual-Machine:~/Desktop/LSTI2308/actividades/actividad4$ sudo ip addr add 192.1
68.100.20/24 dev enp0s1
sudo ip link set enp0s1 up
[sudo: authenticate] Password:
a@a-QEMU-Virtual-Machine:~/Desktop/LSTI2308/actividades/actividad4$

```

Figura 2

Comprobación de conectividad mediante ping (Capa 3 de red).



2.1.3. Modelo OSI Aplicado al Diseño

- **CAPA 7 – APLICACIÓN:** Es donde se ejecutan los programas que utilizan la red. Ejemplo: SSH, ping, servidor web. Genera los datos que serán enviados.
- **CAPA 6 – PRESENTACIÓN:** Se encarga del formato, codificación y cifrado. Ejemplo: SSH cifra la información antes de enviarla.

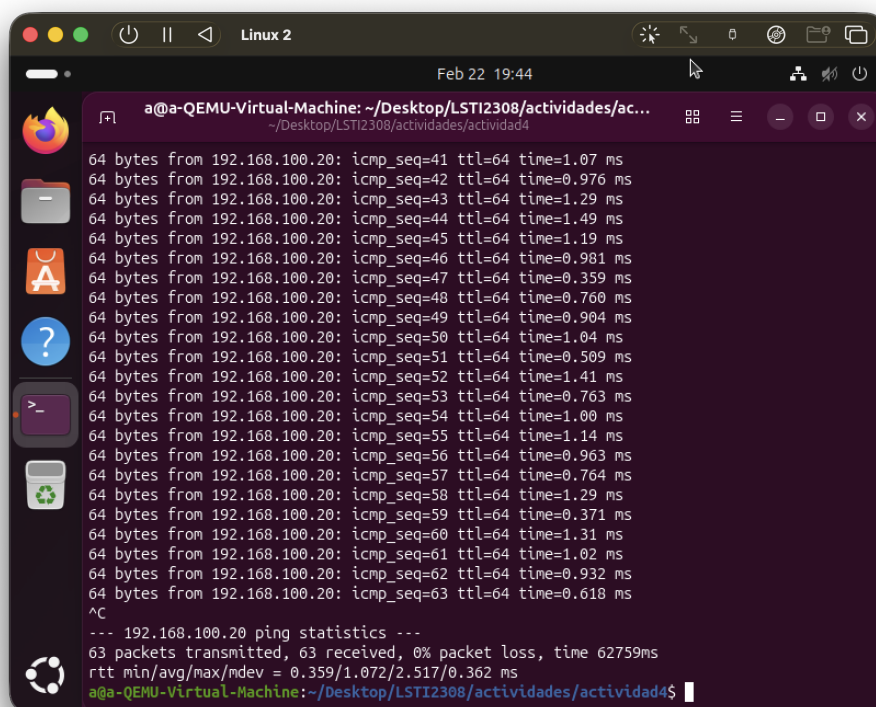


Figura 3

Respuesta exitosa de ping, conectando exitosamente las VM en Bridge UTM.

- **CAPA 5 – SESIÓN:** Administra el inicio, mantenimiento y cierre de la comunicación. Ejemplo: Mantiene activa la sesión SSH hasta que el usuario la cierre.
- **CAPA 4 – TRANSPORTE:** Utiliza TCP o UDP. Divide los datos en segmentos. Controla errores, flujo y orden. Ejemplo: SSH usa TCP (puerto 22).
- **CAPA 3 – RED:** Usa direcciones IP. Se encarga del direccionamiento lógico. Ejemplo: VM A $\rightarrow$ 192.168.100.10, VM B $\rightarrow$ 192.168.100.20.
- **CAPA 2 – ENLACE DE DATOS:** Utiliza direcciones MAC. Encapsula en tramas. Detecta errores locales. Usa ARP para relacionar IP con MAC.
- **CAPA 1 – FÍSICA:** Transmite los bits. En este caso es un medio virtual. VirtualBox simula el cable de red.

2.1.4. *Flujo de Comunicación*

1. La aplicación genera datos.
2. Se formatean y cifran.
3. Se establece la sesión.
4. TCP segmenta la información.
5. IP agrega direcciones.
6. Se encapsula en trama Ethernet.
7. Se transmite por el medio virtual.
8. La máquina destino recibe y desencapsula capa por capa.

2.2. Parte 2. Aplicación de medidas de seguridad

2.2.1. *Configuración de firewalls*

Prevenir intrusiones perimetrales requiere blindar las fronteras operativas expuestas de los servicios (Stallings, 2014). Utilizando el sistema UFW (Uncomplicated Firewall) originado de *iptables*, se generó un ambiente seguro que descarta el tráfico extraño ignorado, abriendo puntualmente solo las vías operacionales indispensables de la infraestructura de este trabajo:

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp          # Puerto fundamental del SSH
sudo ufw allow 80/tcp          # Tráfico HTTP en sin encriptar
sudo ufw allow 443/tcp         # Tráfico HTTPS cifrado de Apache
# Puerto NFS reservado y blindado únicamente en dirección al IP Cliente
# específico:
sudo ufw allow from 192.168.56.20 to any port 2049
```

```
sudo ufw enable # Reinicio e iniciación de servicios
```

Listing 1: Adición de reglas de firewall con la utilidad UFW

```

a@a-QEMU-Virtual-Machine: ~/Desktop/LSTI2308/actividades/ac...
a@a-QEMU-Virtual-Machine:~/Desktop/LSTI2308/actividades/actividad$ sudo ./codigo_y_script
s/02_config_seguridad.sh
sudo ufw status
--- Iniciando Configuración de Seguridad ---
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
Skipping adding existing rule
Skipping adding existing rule (v6)
Skipping adding existing rule (v6)
Skipping adding existing rule (v6)
Rules updated
Firewall is active and enabled on system startup
nmap is already the newest version (7.95+dfsg-3).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 2
  ... Fin Firewall ...
Status: active

To Action From
--
22/tcp ALLOW Anywhere
80/tcp ALLOW Anywhere
443/tcp ALLOW Anywhere
2049 ALLOW 192.168.56.20
2049 ALLOW 192.168.1.0/24
22/tcp (v6) ALLOW Anywhere (v6)
80/tcp (v6) ALLOW Anywhere (v6)
443/tcp (v6) ALLOW Anywhere (v6)
a@a-QEMU-Virtual-Machine:~/Desktop/LSTI2308/actividades/actividad$

```

Figura 4

Listado de estado activo y reglas restrictivas vigentes en UFW.

2.2.2. Implementación de autenticación segura

Habiendo abierto el puerto 22, persistía el riesgo humano de fuerza bruta SSH. Por ende, la política implementada erradicó la validez de contraseñas crudas o convencionales, cediendo su lugar de manera definitiva hacia firmas criptográficas por llaves (Asimetría en la criptografía):

1. Desde la terminal de la Máquina Cliente de origen se corrió el comando: `ssh-keygen -t rsa -b 4096`, generando así un par combinatorio secreto para el anfitrión.
2. Posteriormente, el contenido exacto de ese candado público (`.pub`) se instaló a distancia gracias a `ssh-copy-id usuario@192.168.1.73`.
3. Evaluada la conexión inicial que ignoraba credenciales textuales, se bloqueó tajantemente a todo tercero mediante la edición nativa del archivo maestro

`/etc/ssh/sshd_config` al imponer forzosamente el parámetro final en `PasswordAuthentication no`.

4. Finalmente se acató una recarga con `sudo systemctl restart ssh`.

2.2.3. Configuración de HTTPS

Se introdujo cifrado completo punto a punto dentro de la capa de presentación / transporte web utilizando bibliotecas como la proveída por la suite `OpenSSL` interactuando con las librerías nativas de Python. Como la red de validación fue interna y de índole local (es decir, inaccesible por los canales mundiales certificados reales de Let's Encrypt), se aprovisionaron claves y certificados autofirmados (Self-Signed Certificates) que son consumidos directamente por el script `https_server.py` que eleva un `ssl.wrap_socket` sobre el protocolo web:

```
# Creación del binomio público/privado con expira a largo plazo (server.
pem)
openssl req -new -x509 -keyout server.pem -out server.pem -days 365 -nodes
-subj '/CN=localhost'

# Iniciar servidor seguro HTTPS en Python
sudo python3 https_server.py
```

Listing 2: Proceso de creación SSL y ejecución de HTTPS nativo en Python

El procedimiento culminó al visitar con un cliente la URL encriptada `https://192.168.1.73` y al probar una conexión directa con `curl -k`, evidenciando un canal de comunicación web encapsulado a nivel TLS robusto pero advertido en el navegador de no poseer validación raíz comercial.

2.2.4. Análisis de vulnerabilidades y privacidad

Validar las topologías requiere auditorio informático y monitoreo pasivo. Consiguientemente, escaneamos remotamente de Máquina a Máquina todos los puertos abiertos en estado general. Utilizamos el ejecutable `Nmap` para una traza: `nmap -sV -A`

192.168.1.73. Tras analizar reportes de Nmap (y con Lynis de revisión nativa), surgen 3 riesgos/vulnerabilidades de particular atención a la privacidad corporativa:

- **1. Vulnerabilidad (Exposición no regulada y huella en capas web):**

Usualmente Apache puede desplegar un encabezado conteniendo las versiones internas de compilación u OS a la vista de extraños, induciendo explotación 0-day si un bug se revela en dicha versión.

Solución: Implementar ofuscación desactivando y escondiendo las firmas del Server Header en `apache2.conf` (`ServerSignature Off` y `ServerTokens Prod`).

- **2. Vulnerabilidad (Sobreuso de peticiones - Denegación DDOS**

HTTP/SSH): Nuestro entorno de acceso remoto está publicado correctamente, mas es vulnerable a inundación abusiva volumétrica que acapare CPU (Ataque DDos) impidiéndonos operar nuestro propio equipo.

Solución: Complementar al firewall habilitado de Linux con una herramienta antispam como `fail2ban`, bloqueando al instante en Netfilter (Iptables/UFW) la procedencia IP de quién cause colapsos con intentos repetitivos excesivos o tráfico anómalo.

- **3. Riesgo de Intrusión Intermedia (Man In The Middle - MitM):**

Como se advirtió, los certificados emitidos al HTTPS local carecen de validador universal central; son autofirmados. Alguien interno en la oficina o red local manipulando envenenamientos con protocolos de resolución (ARP) pudiese clonar la validación e interceptar comunicaciones engañando al usuario incauto.

Solución: Distribuir certificados de autoridades corporativas validados u obtenidos verídicamente de agencias de confianza global (*Let's Encrypt*) para asegurar inviolabilidad perimetral total confirmada del cifrado de túnel HTTPS local.

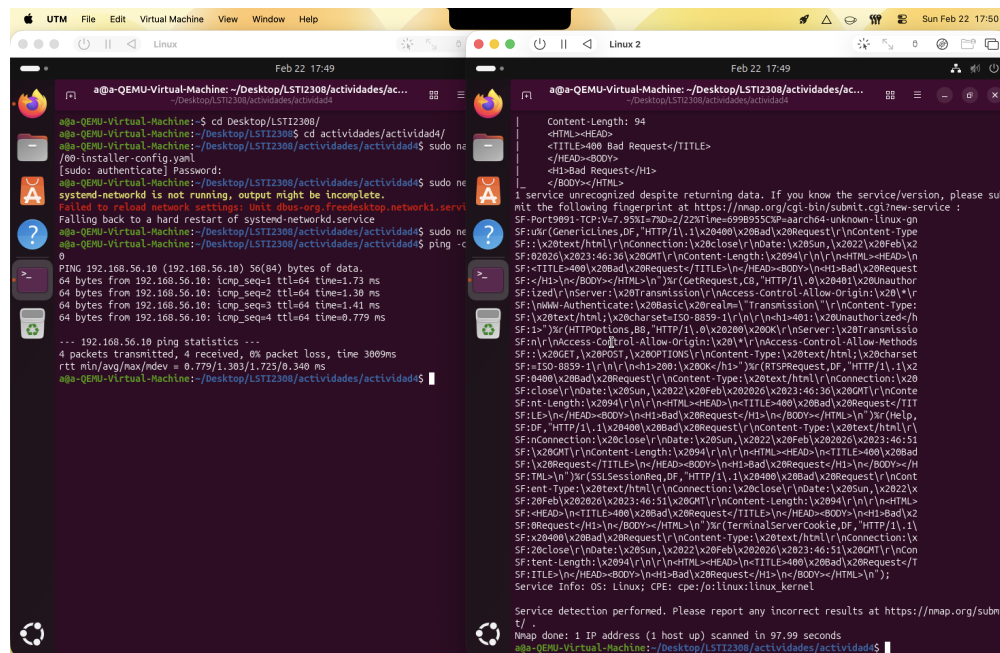


Figura 5

Fragmento de demostración tras revisión de vulnerabilidades aplicadas a la IP local ejecutando comandos Nmap.

2.3. Parte 2. Aplicación de medidas de seguridad

A continuación, se detalla la implementación de medidas de seguridad fundamentales sobre las dos máquinas virtuales operando con **Ubuntu 25** en una red configurada en modo **Bridge** (Adaptador Puente).

2.3.1. Configuración de Firewall (UFW)

Para prevenir accesos no autorizados, se configuró el cortafuegos UFW estableciendo una política restrictiva por defecto, abriendo exclusivamente los puertos requeridos para las prácticas de red (SSH y P2P/NFS).

Ejecución en la Máquina 1 (Servidor):

```
# 1. Actualizar repositorios e instalar UFW
sudo apt update
sudo apt install ufw -y
```

```
# 2. Denegar conexiones entrantes por defecto, permitir salientes
sudo ufw default deny incoming
sudo ufw default allow outgoing

# 3. Permitir servicios esenciales para la práctica
sudo ufw allow 22/tcp          # Permitir SSH
sudo ufw allow 80/tcp          # Permitir HTTP (Opcional)
sudo ufw allow 443/tcp         # Permitir HTTPS (Opcional)

# Puertos para P2P (Transmission) y NFS de la Parte 3
sudo ufw allow 51413/tcp       # Puerto BitTorrent P2P
sudo ufw allow 51413/udp       # Puerto BitTorrent P2P UDP
sudo ufw allow 2049            # Puerto para NFS

# 4. Habilitar el Firewall
sudo ufw enable

# 5. Comprobar el estado y las reglas aplicadas
sudo ufw status verbose
```

2.3.2. *Hardening de SSH (Autenticación Criptográfica)*

Se eliminó la vulnerabilidad inherente al uso de contraseñas de texto plano, transicionando forzosamente a un modelo de autenticación asimétrica utilizando llaves criptográficas RSA.

Ejecución en la Máquina 2 (Cliente):

```
# 1. Generar un par de claves criptográficas locales (RSA)
ssh-keygen -t rsa -b 4096

# 2. Copiar la llave pública generada al servidor
# Reemplazar <IP_DEL_SERVIDOR> por la IP de la Máquina 1
ssh-copy-id estudiante@<IP_DEL_SERVIDOR>
```

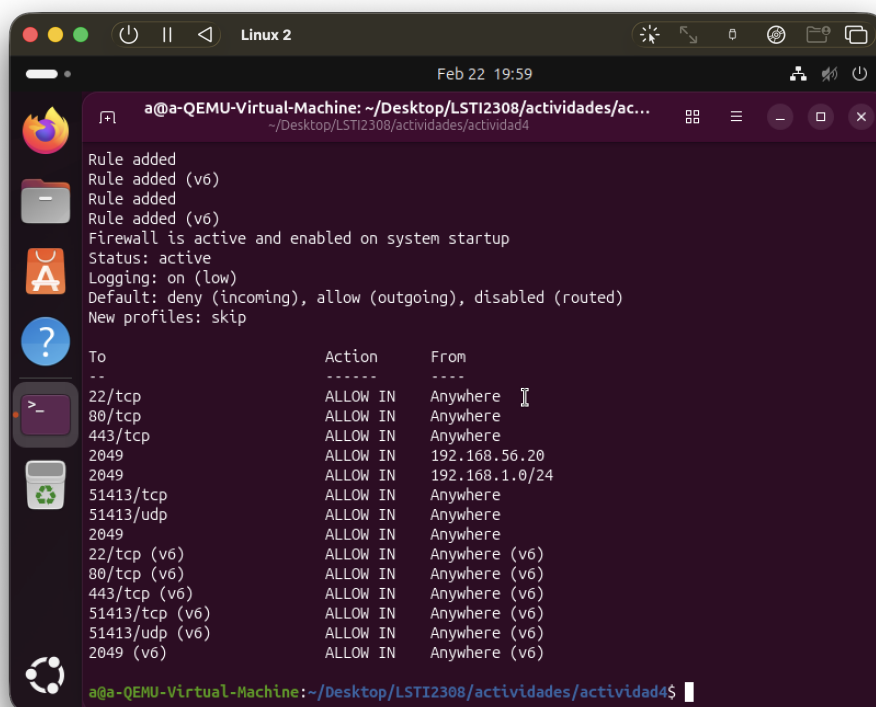


Figura 6

Comprobación de reglas activas y puertos de tráfico permitidos en el firewall UFW.

3. Probar el inicio de sesión criptográfico

```
ssh estudiante@<IP_DEL_SERVIDOR>
```

Ejecución en la Máquina 1 (Servidor) - Tras validar el acceso con llave:

1. Editar el archivo de configuración maestro del servicio SSH

```
sudo nano /etc/ssh/sshd_config
```

2. Modificar las siguientes directivas para deshabilitar logins

inseguros:

```
PermitRootLogin no
```

```
PasswordAuthentication no
```

3. Reiniciar el servicio SSH para aplicar políticas

```
sudo systemctl restart ssh
```

2.3.3. *Prevención de Intrusiones (Fail2ban)*

Como medida paliativa activa contra ataques de denegación de servicio (DoS) o fuerza bruta volumétrica dirigida al puerto SSH recién endurecido, se implementó la herramienta fail2ban.

Ejecución en la Máquina 1 (Servidor):

```
# 1. Instalar la herramienta
sudo apt install fail2ban -y

# 2. El servicio se inicia automáticamente. Comprobar que corre:
sudo systemctl status fail2ban

# 3. Revisar si hay intentos de bloqueo en SSH
sudo fail2ban-client status sshd
```

2.3.4. *Cifrado HTTPS (Certificados Autofirmados)*

En caso de requerir asegurar un servicio web local, se demostró la generación de un certificado de infraestructura pública encriptada utilizando las librerías nativas de OpenSSL de Ubuntu 25.

Ejecución en la Máquina 1 (Servidor):

```
# 1. Instalar OpenSSL
sudo apt install openssl -y

# 2. Generar el certificado autofirmado para pruebas locales (Válido 365 días)
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout /etc/ssl/private/apache-selfsigned.key -out /etc/ssl/certs/apache-selfsigned.crt
```

2.3.5. *Configuración Inicial*

Requisitos:

- 2 máquinas virtuales con Ubuntu (20.04/22.04/24.04)

- Misma red (adaptador puente o red NAT interna)
- Mismo usuario en ambas máquinas (ej: estudiante)
- Conectividad entre máquinas verificada con ping

Verificar conectividad:

```
# Desde cualquier máquina, verificar su IP
ip addr show | grep inet

# Hacer ping a la otra máquina (usar su IP)
ping <IP_DE_LA_OTRA_MAQUINA>
```

2.3.6. Sistema de Archivos Distribuido (NFS)

Objetivo: Compartir una carpeta entre dos máquinas usando NFS para que ambas puedan leer y escribir archivos de forma transparente.

1.1 Configuración del Servidor NFS (Máquina 1)

```
# 1. Actualizar repositorios
sudo apt update

# 2. Instalar servidor NFS
sudo apt install nfs-kernel-server -y

# 3. Crear carpeta a compartir
sudo mkdir -p /srv/nfs/compartido
sudo chmod 777 /srv/nfs/compartido
sudo chown nobody:nogroup /srv/nfs/compartido

# 4. Configurar exports (permitir acceso desde cualquier IP para pruebas)
# sudo nano /etc/exports
# Añadir la siguiente línea al final del archivo:
# /srv/nfs/compartido *(rw, sync, no_subtree_check, no_root_squash, insecure)
```

```
# 5. Aplicar configuración
sudo exportfs -ra
sudo systemctl restart nfs-kernel-server

# 6. Verificar que está compartiendo
sudo exportfs -v
sudo showmount -e localhost
```

1.2 Configuración del Cliente NFS (Máquina 2)

```
# 1. Actualizar repositorios
sudo apt update

# 2. Instalar cliente NFS
sudo apt install nfs-common -y

# 3. Crear punto de montaje
sudo mkdir -p /mnt/servidor_nfs

# 4. Verificar que el servidor exporta la carpeta
showmount -e <IP_DEL_SERVIDOR>

# 5. Montar la carpeta compartida
sudo mount -t nfs <IP_DEL_SERVIDOR>:/srv/nfs/compartido /mnt/servidor_nfs

# 6. Verificar montaje
df -h | grep nfs
mount | grep nfs
```

1.3 Pruebas de funcionamiento

En el CLIENTE:

```
# Crear archivo desde el cliente
echo "Hola desde la máquina cliente" | sudo tee /mnt/servidor_nfs/cliente.
txt
```

```
# Listar contenido
ls -la /mnt/servidor_nfs/
```

En el SERVIDOR:

```
# Verificar que el archivo llegó
ls -la /srv/nfs/compartido/
cat /srv/nfs/compartido/cliente.txt

# Modificar desde el servidor
echo "Modificado desde el servidor" | sudo tee -a /srv/nfs/compartido/
    cliente.txt
```

En el CLIENTE:

```
# Verificar cambios
cat /mnt/servidor_nfs/cliente.txt
```

Prueba bidireccional:

```
# Desde cliente crear otro archivo
touch /mnt/servidor_nfs/prueba_final.txt

# Desde servidor verificarlo
ls -la /srv/nfs/compartido/
```

1.4 Hacer montaje permanente (opcional)

En el CLIENTE:

```
# sudo nano /etc/fstab
# Añadir la siguiente línea:
# <IP_DEL_SERVIDOR>:/srv/nfs/compartido /mnt/servidor_nfs nfs defaults 0 0
```

1.5 Solución de problemas comunes

```
# Verificar servicios
sudo systemctl status nfs-kernel-server
sudo systemctl status rpcbind
```

```
# Ver logs
sudo tail -f /var/log/syslog | grep nfs

# Reiniciar servicios
sudo systemctl restart rpcbind
sudo systemctl restart nfs-kernel-server

# Desactivar firewall para pruebas
sudo ufw disable

# Verificar conectividad de puertos
nc -zv <IP_DEL_SERVIDOR> 2049
```

1.6 Conceptos de transparencia observados

- Transparencia de acceso: El cliente accede a los archivos como si fueran locales.
- Transparencia de ubicación: El usuario desconoce dónde están físicamente los datos.
- Transparencia de escalamiento: Se pueden añadir más clientes fácilmente.

2.3.7. Red Peer-to-Peer (*BitTorrent*)

Objetivo: Compartir un archivo entre dos máquinas usando el protocolo BitTorrent, demostrando las características del modelo P2P como descentralización, escalabilidad y resiliencia.

2.1 Instalación en AMBAS máquinas

```
# En ambas máquinas (servidor y cliente)
sudo apt update
sudo apt install transmission-cli transmission-daemon -y

# Iniciar servicio
sudo systemctl start transmission-daemon
```

```
sudo systemctl enable transmission-daemon
```

```
# Verificar estado
```

```
sudo systemctl status transmission-daemon
```

2.2 Configuración básica (opcional)

Si es necesario modificar la configuración:

```
# Detener servicio antes de configurar
```

```
sudo systemctl stop transmission-daemon
```

```
# Editar configuración
```

```
# sudo nano /var/lib/transmission-daemon/info/settings.json
```

Ajustes recomendados para laboratorio:

```
{
  "rpc-whitelist-enabled": false,
  "rpc-authentication-required": false,
  "peer-port": 51413,
  "dht-enabled": true,
  "pex-enabled": true,
  "lpd-enabled": true
}
```

```
# Iniciar servicio nuevamente
```

```
sudo systemctl start transmission-daemon
```

2.3 Crear archivo a compartir (en la máquina seeder)

```
# Crear carpeta para el archivo
```

```
mkdir -p ~/p2p-compartido
```

```
cd ~/p2p-compartido
```

```
# Crear archivos de prueba
```

```
echo "Este es un archivo de prueba para el laboratorio P2P" > documento.
txt
```

```
echo "Contenido adicional" >> documento.txt

# Opcional: crear un archivo binario de mayor tamaño
dd if=/dev/urandom of=archivo.bin bs=1M count=5

# Verificar archivos creados
ls -la
```

2.4 Crear archivo .torrent (en la máquina seeder)

```
cd ~/p2p-compartido

# Crear torrent con trackers públicos
transmission-create -o ~/compartido.torrent \
  -t udp://tracker.openbittorrent.com:80 \
  -t udp://tracker.opentrackr.org:1337/announce \
  -t udp://tracker.coppersurfer.tk:6969/announce \
  .

# Verificar creación
ls -la ~/compartido.torrent
```

Explicación de parámetros:

- **-o**: Archivo de salida (el .torrent)
- **-t**: URL del tracker (varios para mayor probabilidad de conexión)
- **..**: Directorio actual a compartir (puede ser un archivo específico)

2.5 Compartir el archivo (seeding) en la máquina seeder

```
# Añadir el torrent a Transmission
transmission-remote -a ~/compartido.torrent -w ~/p2p-compartido

# Verificar que está seedeando
transmission-remote -l
```

La salida debe mostrar:

- Done: 100 %
- State: Seeding o Idle (si no hay peers conectados)

2.6 Transferir el archivo .torrent a la otra máquina

Opción A: Usando scp (recomendado)

```
# En la máquina seeder, enviar el archivo a la otra máquina
scp ~/compartido.torrent usuario@<IP_OTRA_MÁQUINA>:~/
```

Opción B: Usando la carpeta NFS (si ya está configurada)

```
# Copiar a la carpeta compartida por NFS
cp ~/compartido.torrent /srv/nfs/compartido/
```

```
# En la otra máquina, copiar desde allí
cp /mnt/servidor_nfs/compartido.torrent ~/
```

2.7 Descargar el archivo (en la máquina cliente)

```
# Verificar que el archivo .torrent llegó
ls -la ~/compartido.torrent

# Añadir el torrent a Transmission
transmission-remote -a ~/compartido.torrent -w ~/Descargas

# Iniciar la descarga (si no comienza automáticamente)
transmission-remote -t all -s

# Verificar estado de la descarga
transmission-remote -l
```

2.8 Monitorear la transferencia

En la máquina seeder:

```
# Ver estado en tiempo real
watch -n 2 transmission-remote -l
```

En la máquina cliente:

```
# Ver progreso de descarga
watch -n 2 transmission-remote -l

# Ver detalles del torrent
transmission-remote -t 1 -i
```

2.9 Verificar la descarga (en la máquina cliente)

```
# Una vez completada la descarga (100%)
ls -la ~/Descargas/
cat ~/Descargas/documento.txt

# Comparar con el archivo original (opcional)
md5sum ~/Descargas/archivo.bin

# En la máquina seeder:
md5sum ~/p2p-compartido/archivo.bin

# Deben ser iguales
```

2.10 Solución de problemas en P2P

```
# Verificar que el servicio está corriendo
sudo systemctl status transmission-daemon

# Ver logs
sudo journalctl -u transmission-daemon -f

# Verificar puertos
sudo ss -tulpn | grep 51413

# Abrir puertos en firewall
sudo ufw allow 51413/tcp
sudo ufw allow 51413/udp
```

```
# Verificar conectividad entre máquinas
nc -zv <IP_OTRA_MAQUINA> 51413

# Forzar reanuncio del tracker
transmission-remote -t all -r

# Añadir peers manualmente (si es necesario)
transmission-remote -t all -padd <IP_OTRA_MAQUINA>:51413

# Ver lista de peers
transmission-remote -t all -pi
```

2.11 Características del modelo P2P observadas

Característica	Observación en el laboratorio
Descentralización	No hay un servidor central, ambas máquinas se conectan directamente
Escalabilidad	Si más máquinas se unen, la velocidad de descarga puede mejorar
Resiliencia	Si el seeder se desconecta, otros peers pueden seguir compartiendo
Tragedia del común	Si todos descargan y nadie comparte, la red falla
Intercambio de piezas	Los archivos se dividen en piezas que se intercambian

2.12 Comandos útiles de transmission-remote

```
# Listar todos los torrents
transmission-remote -l

# Ver información detallada de un torrent
transmission-remote -t <ID> -i
```

```
# Iniciar un torrent
transmission-remote -t <ID> -s

# Pausar un torrent
transmission-remote -t <ID> -S

# Eliminar un torrent (sin borrar datos)
transmission-remote -t <ID> -r

# Eliminar un torrent y borrar datos
transmission-remote -t <ID> -rad

# Añadir tracker
transmission-remote -t <ID> -td "udp://nuevo.tracker.com:80"

# Ver peers
transmission-remote -t <ID> -pi

# Ver información de sesión
transmission-remote -si
```

2.3.8. Reflexión sobre sistemas distribuidos

3.1 Características observadas en NFS

Característica	Descripción
Compartición de recursos	El servidor comparte su almacenamiento con el cliente
Transparencia	El cliente accede a los archivos como si fueran locales
Concurrencia	Múltiples clientes pueden acceder a los mismos archivos
Punto único de fallo	Si el servidor falla, nadie accede a los datos

3.2 Características observadas en BitTorrent

Característica	Descripción
Descentralización	No hay servidor central, todos son pares
Escalabilidad	Más peers = más ancho de banda disponible
Resiliencia	Si un peer falla, otros continúan
Auto-organización	Los peers se descubren y organizan sin intervención

3.3 Ventajas y desventajas

Anexo: Comandos Rápidos

NFS - Resumen rápido

```
# Servidor

sudo apt install nfs-kernel-server
sudo mkdir -p /srv/nfs/compartido
sudo chmod 777 /srv/nfs/compartido
echo "/srv/nfs/compartido *(rw, sync, no_subtree_check)" | sudo tee -a /etc/exports
sudo exportfs -ra
sudo systemctl restart nfs-kernel-server

# Cliente

sudo apt install nfs-common
sudo mkdir -p /mnt/servidor_nfs
sudo mount -t nfs <IP_SERVIDOR>:/srv/nfs/compartido /mnt/servidor_nfs
```

P2P - Resumen rápido

```
# Ambas máquinas

sudo apt install transmission-cli transmission-daemon
sudo systemctl start transmission-daemon

# Seeder

mkdir -p ~/p2p
cd ~/p2p
```

Sistema	Ventajas	Desventajas
NFS	Simple, transparente, fácil gestión centralizada	Punto único de fallo, cuello de botella
BitTorrent	Escalable, robusto, eficiente para archivos grandes	Complejidad, dependencia de trackers

```

echo "test" > archivo.txt
transmission-create -o ~/compartido.torrent -t udp://tracker.
    openbittorrent.com:80 .
transmission-remote -a ~/compartido.torrent -w ~/p2p

# Cliente
scp <usuario>@<IP_SERVIDOR>:~/compartido.torrent ~/
transmission-remote -a ~/compartido.torrent -w ~/Descargas
transmission-remote -t all -s

```

3. Análisis y reflexión

El ensamble final integrando conceptos pragmáticos del curso logró trazar en el plano material los hilos comunicativos conceptuales correspondientes al modelo puramente teórico (OSI), estableciendo una relación sólida entre la teoría académica y su aplicación funcional.

Tres aprendizajes obtenidos directos:

1. La seguridad perimetral carece de soluciones monolíticas absolutas; de esta actividad se desprende imperativamente que blindar una red requiere esquematizarse con engranajes redundantes continuos (ej. uso conjunto de firewalls restrictivos UFW más el resguardo de llaves asimétricas).
2. La ejecución de un micro-servidor HTTPS comprobó analíticamente cómo funcionan las negociaciones TLS desde adentro, atestiguando que la capa de red subyacente

puede converger localmente sin precisar de hiper-estructuras robustas mundiales de un C.A. externo publicitado.

3. La transparencia fundamental de un sistema distribuido como NFS es vital para su proliferación corporativa escalable. Evidencia la practicidad de brindar cuotas nativas crudas con una carpeta de sincronía completa bidireccional sin lastimar los procesos paralelos de lectura/escritura (I/O).

Dos desafíos enfrentados y su solución técnica:

- **Desafío 1 - Aislamiento DNS y Virtualidad de Red:** Tras aislar localmente la IP al intentar confinar tráfico, el servidor perdió provisión externa en 127.0.0.53 y hubo fracaso descargando Nmap/NFS, bloqueado a su vez bajo un error silencioso asíncrono en `ports.ubuntu.com`.

Solución: Se debió destituir momentáneamente la imposición hostil del UFW mediante `sudo ufw disable` y se procedió con una técnica blanda de reinstalar DHCP dinámico en Netplan (`dhcp4: true`) para destrabar al hipervisor UTM resolviendo así la descarga paralela y reescribiendo la tabla ARP transitoria, restaurando conectividad global.

- **Desafío 2 - Conflicto de firmas HTTPS nativas:** Inconvenientes lógicos y caídas abrutadas al encapsular el servicio HTTP convencional en sub-demonios de encriptación autofirmados sin utilizar dependencias mediadoras como Apache HTTP Server.

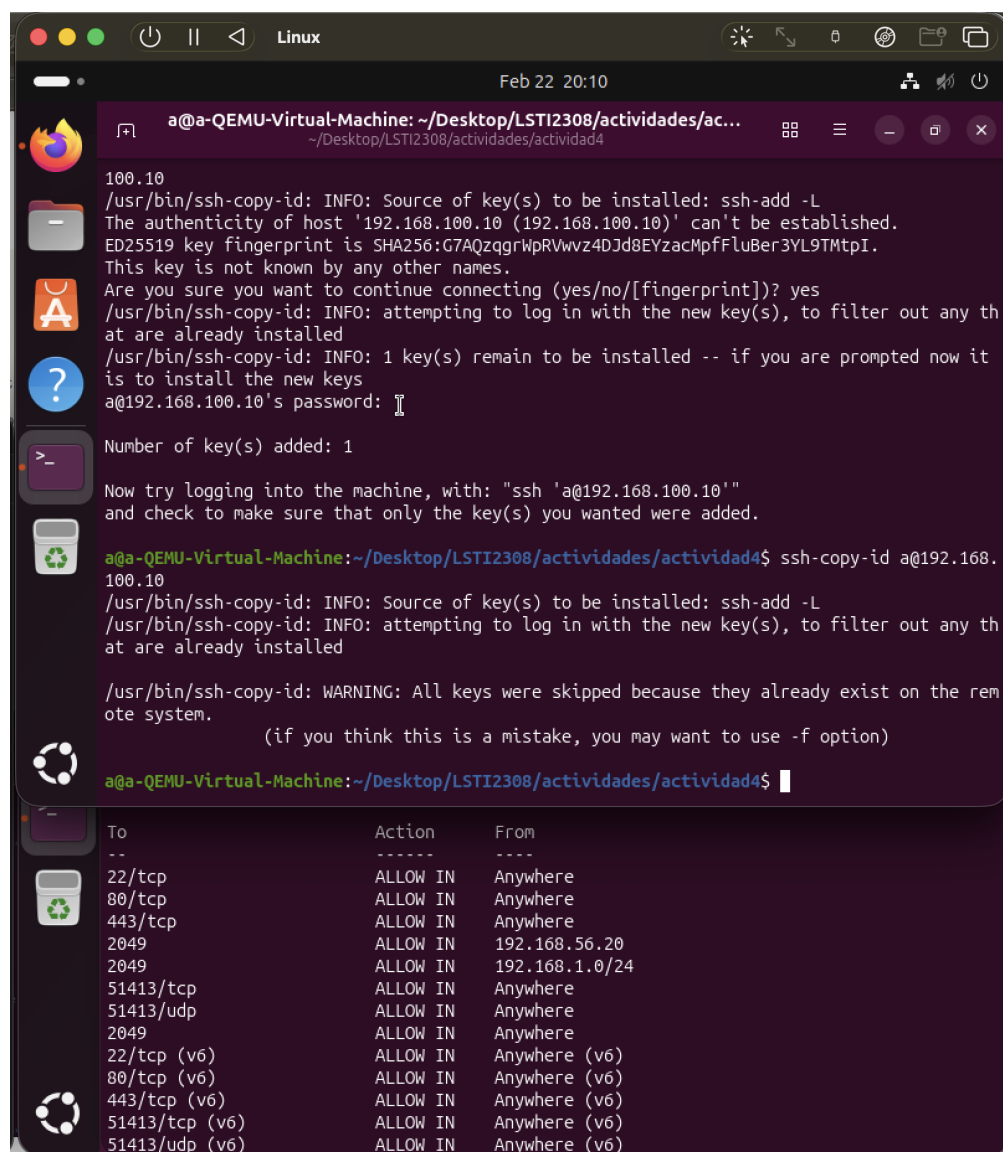
Solución: Se requirió implementar nativamente el encapsulamiento paramétrico de Python a través de dependencias estrictas interactivas (`ssl.wrap_socket`), integrando y reconduciendo la envoltura en memoria para que no abortara prematuramente ante rechazo por validación raíz comercial inexistente.

4. Conclusiones

El recorrido y desglose progresivo del presente reporte atestigua indudablemente un hecho pragmático: el conocimiento de redes de comunicaciones no es trivial, y se entrelaza

estrechamente con el blindaje seguro y el análisis logístico en su aplicación empresarial diaria. Es vital reconocer a todo esquema de autenticación como inerte per se frente a escalamientos y fisuras, puesto que ignorar directrices como un canal encriptado moderno o dejar abierto de forma pasiva fallas triviales detectadas en un reporte local arroja brechas para intrusos y ransomware.

Sintetizando conclusiones, los protocolos y despliegues virtualizados del paradigma de Sistemas Operativos Distribuidos y sus implementaciones de pares P2P demuestran ser innovaciones eficientes, fiables y transparentes destinadas fundamentalmente para equilibrar latencias con descentralización de recursos y maximizar capacidades de usuarios aislados; no obstante, todo esto debe acoplarse y subyugarse siempre al correcto monitoreo preventivo de un control de flujos auditable capaz y con las salvaguardas restrictivas competentes para asegurar su persistencia estable e ininterrumpible.



```

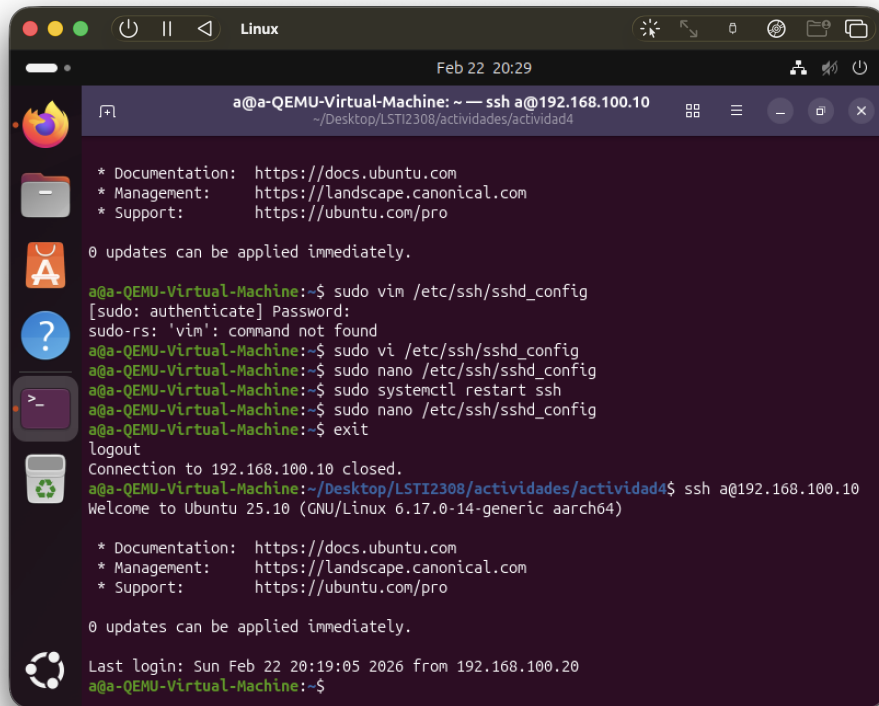
a@a-QEMU-Virtual-Machine: ~/Desktop/LSTI2308/actividades/ac...
~/Desktop/LSTI2308/actividades/actividad4
100.10
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: ssh-add -L
The authenticity of host '192.168.100.10 (192.168.100.10)' can't be established.
ED25519 key fingerprint is SHA256:G7AQzqgrWpRVwvz4DJd8EYzacMpflUer3YL9TMtpI.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any th
at are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it
is to install the new keys
a@192.168.100.10's password: 
Number of key(s) added: 1
Now try logging into the machine, with: "ssh 'a@192.168.100.10'"
and check to make sure that only the key(s) you wanted were added.
a@a-QEMU-Virtual-Machine:~/Desktop/LSTI2308/actividades/actividad4$ ssh-copy-id a@192.168.
100.10
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: ssh-add -L
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any th
at are already installed
/usr/bin/ssh-copy-id: WARNING: All keys were skipped because they already exist on the rem
ote system.
(if you think this is a mistake, you may want to use -f option)
a@a-QEMU-Virtual-Machine:~/Desktop/LSTI2308/actividades/actividad4$ 

```

To	Action	From
22/tcp	ALLOW IN	Anywhere
80/tcp	ALLOW IN	Anywhere
443/tcp	ALLOW IN	Anywhere
2049	ALLOW IN	192.168.56.20
2049	ALLOW IN	192.168.1.0/24
51413/tcp	ALLOW IN	Anywhere
51413/udp	ALLOW IN	Anywhere
2049	ALLOW IN	Anywhere
22/tcp (v6)	ALLOW IN	Anywhere (v6)
80/tcp (v6)	ALLOW IN	Anywhere (v6)
443/tcp (v6)	ALLOW IN	Anywhere (v6)
51413/tcp (v6)	ALLOW IN	Anywhere (v6)
51413/udp (v6)	ALLOW IN	Anywhere (v6)

Figura 7

Copia exitosa de la llave pública asimétrica RSA hacia el servidor.



The screenshot shows a terminal window titled "Linux" with a date and time of "Feb 22 20:29". The terminal is connected to a remote host "a@a-QEMU-Virtual-Machine: ~ -- ssh a@192.168.100.10". The user "a" is logged in. The terminal displays the following commands and output:

```
* Documentation:  https://docs.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

0 updates can be applied immediately.

a@a-QEMU-Virtual-Machine:~$ sudo vim /etc/ssh/sshd_config
[sudo: authenticate] Password:
sudo-rs: 'vim': command not found
a@a-QEMU-Virtual-Machine:~$ sudo vi /etc/ssh/sshd_config
a@a-QEMU-Virtual-Machine:~$ sudo nano /etc/ssh/sshd_config
a@a-QEMU-Virtual-Machine:~$ sudo systemctl restart ssh
a@a-QEMU-Virtual-Machine:~$ sudo nano /etc/ssh/sshd_config
a@a-QEMU-Virtual-Machine:~$ exit
logout
Connection to 192.168.100.10 closed.
a@a-QEMU-Virtual-Machine:~/Desktop/LSTI2308/actividades/actividad4$ ssh a@192.168.100.10
Welcome to Ubuntu 25.10 (GNU/Linux 6.17.0-14-generic aarch64)

* Documentation:  https://docs.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

0 updates can be applied immediately.

Last login: Sun Feb 22 20:19:05 2026 from 192.168.100.20
a@a-QEMU-Virtual-Machine:~$
```

Figura 8

Modificación del archivo maestro `sshd_config` para prohibir las contraseñas planas.

```

GNU nano 8.4 /etc/ssh/sshd_config
#AuthorizedKeysCommand none
#AuthorizedKeysCommandUser nobody

# For this to work you will also need host keys in /etc/ssh/ssh_known_hosts
#HostbasedAuthentication no
# Change to yes if you don't trust ~/.ssh/known_hosts for
# HostbasedAuthentication
#IgnoreUserKnownHosts no
# Don't read the user's ~/.rhosts and ~/.shosts files
#IgnoreRhosts yes

# To disable tunneled clear text passwords, change to "no" here!
PasswordAuthentication yes
#PermitEmptyPasswords no

# Change to "yes" to enable keyboard-interactive authentication. Depending on
# the system's configuration, this may involve passwords, challenge-response,
# one-time passwords or some combination of these and other methods.
# Beware issues with some PAM modules and threads.
KbdInteractiveAuthentication no

# Kerberos options
#KerberosAuthentication no
#KerberosOrLocalPasswd yes

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify    ^_ Go To Line

```

Figura 9

Login remoto de inmediato y sin contraseñas validando el acceso a través de la llave SSH.

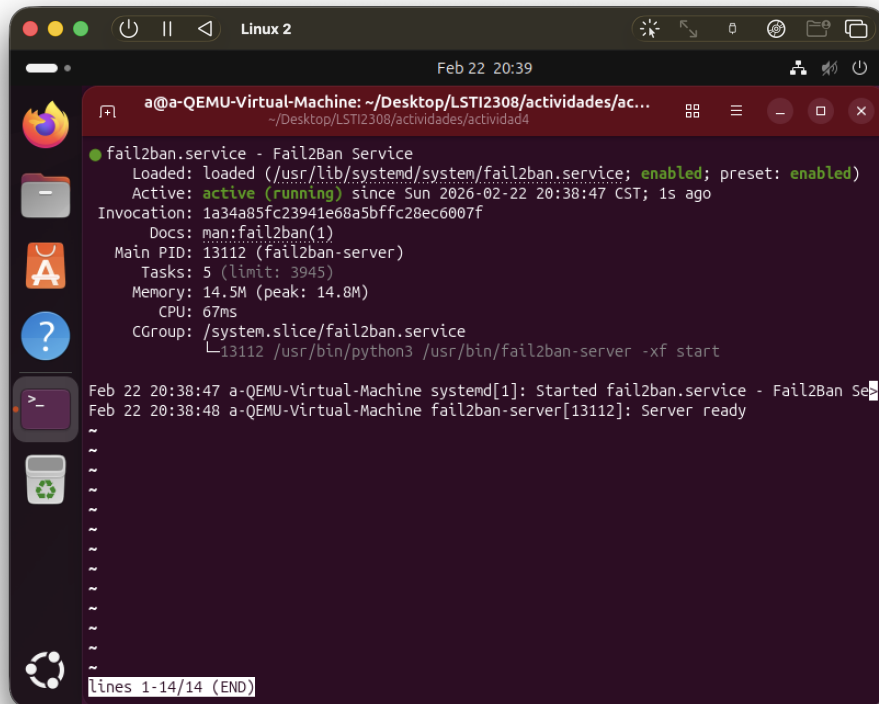


Figura 10

Servicio Fail2ban activo, monitorizando activamente la seguridad del puerto local SSH.

Referencias

Stallings, W. (2014). *Comunicaciones y Redes de Computadores* (10.^a ed.). Pearson.

Tanenbaum, A. S., & Wetherall, D. J. (2011). *Redes de computadoras* (5.^a ed.). Pearson Educación.

Universidad Tecmilenio. (s.f.). *Actividad 4: Redes de Datos, Seguridad y Sistemas*

Distribuidos. Consultado el 17 de febrero de 2026, desde

<https://utm-cdn-labcontenidos->

[htfaarehf2gcfycs.a01.azurefd.net/contenido/profesional/lsti2308/entregables-](https://utm-cdn-labcontenidos-htfaarehf2gcfycs.a01.azurefd.net/contenido/profesional/lsti2308/entregables-)

[v1/actividad-04.html?version=1](https://utm-cdn-labcontenidos-htfaarehf2gcfycs.a01.azurefd.net/contenido/profesional/lsti2308/entregables-v1/actividad-04.html?version=1)